Transformers and Transformer Architecture

Abstract

Transformers are neural network architectures that have transformed natural language processing and generative artificial intelligence. Introduced by Vaswani and colleagues in 2017, they use attention to connect information across token positions without relying on recurrent processing. This report explains Transformer foundations, the encoder–decoder design, input preprocessing, tokenization, embeddings, positional encoding, self-attention and multi-head attention, and the role of normalization and feed-forward layers. It also covers encoder-only, decoder-only, and encoder–decoder model families, pretraining versus fine-tuning, efficient adaptation methods, training and inference, applications, and limitations.

1. Introduction

A Transformer is a deep neural network that learns relationships among components of an input by computing attention-based representations. Originally developed for sequence-to-sequence translation, Transformers now underpin a wide range of language, vision, audio, and multimodal systems. Unlike recurrent neural networks, which update hidden states sequentially, Transformer layers can represent interactions among many input tokens concurrently during training. This ability facilitates accelerator parallelism, although autoregressive decoding still produces new tokens in order.

Consider “The student placed the book on the table because it was heavy.” A model needs contextual information to associate the pronoun “it” with a plausible referent. Self-attention makes it possible to weight relevant positions dynamically. Such weights are learned statistical interactions, not guaranteed grammatical explanations.

2. From RNNs and LSTMs to Transformers

Recurrent Neural Networks (RNNs) process a sequence step by step, using a hidden state to carry information from earlier positions. Long Short-Term Memory networks (LSTMs) improve this design with gating operations that help retain useful information. Nevertheless, their sequential processing limits parallelism and can complicate optimization over long sequences. Transformers instead use attention layers, feed-forward transformations, and positional information, enabling more direct communication between distant tokens.

Property	RNN / LSTM	Transformer
Sequence processing	Sequential state updates	Parallel token processing during training
Long-distance interaction	Passed through recurrent states	Attention connects positions directly
Order awareness	Built into recurrence	Positional representations are supplied
Main computation	Recurrence and gates	Attention and feed-forward blocks

3. Overall Transformer Architecture

The original Transformer has an encoder stack and a decoder stack. Its encoder constructs contextual representations of the source sequence; its decoder predicts target tokens while conditioning on previously generated targets and the encoder output. The original base model used six encoder layers and six decoder layers, with model dimension 512 and eight attention heads (Vaswani et al., 2017).

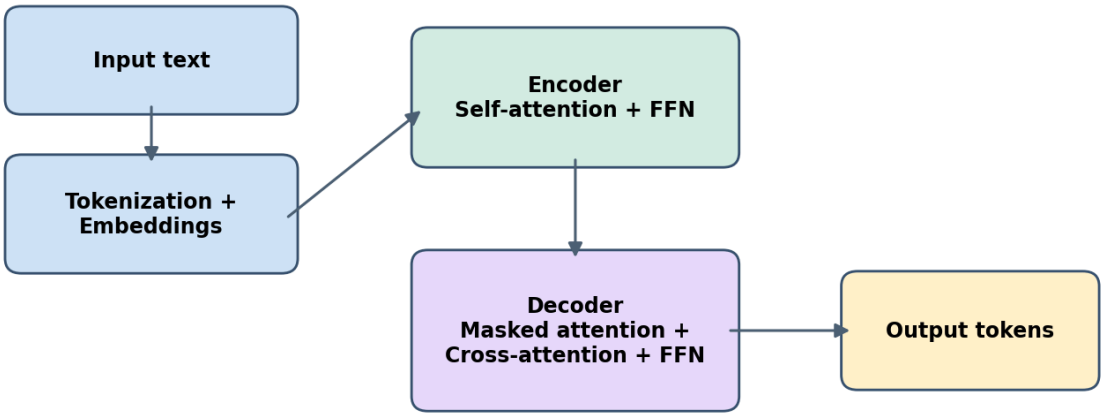


Figure 1. Simplified encoder–decoder architecture, with cross-attention from decoder to encoder.

3.1 Encoder

Each encoder layer consists of multi-head self-attention followed by a position-wise feed-forward network. Residual connections and layer normalization surround these operations. In bidirectional encoder attention, a token can incorporate information from other unmasked input positions, producing contextual vectors useful for comprehension tasks.

3.2 Decoder

The original decoder contains masked self-attention, encoder–decoder cross-attention, and a feed-forward network. A causal mask prevents a target token from attending to future target tokens. Cross-attention draws keys and values from encoder outputs, while queries originate from the decoder state. A final vocabulary projection and softmax assign probabilities to candidate next tokens.

4. Tokenization

Tokenization converts text into discrete units and then vocabulary IDs. Word-level tokenization is simple but yields large vocabularies and difficulties with rare words. Character-level tokenization avoids unknown words but lengthens sequences. Subword methods—such as Byte Pair Encoding (BPE), WordPiece, and Unigram—split uncommon words into smaller reusable fragments while preserving common units. Exact splits are tokenizer-dependent.

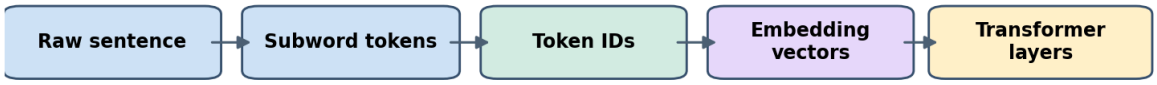


Figure 2. Text is tokenized into IDs, mapped to vectors, and processed by Transformer layers.

For illustration, “unbelievable” might be split as “un”, “believ”, and “able”; these are hypothetical tokens, not an assertion about any specific tokenizer. Special tokens serve model-specific purposes: BERT uses tokens such as [CLS] and [SEP], and generative models use end-of-sequence markers. Padding aligns sequence lengths within batches; attention masks prevent padding positions from affecting computations. Truncation controls input length but can remove meaningful information.

5. Token Embeddings

A token embedding is a dense learned vector indexed by its vocabulary ID. The embedding table E has shape V × d_model, where V is vocabulary size and d_model is representation dimension. For example, a vocabulary with 30,000 entries and 512-dimensional vectors would use an embedding matrix with 30,000 rows and 512 columns. Initial embeddings are not the same as contextual outputs: subsequent Transformer layers adjust each token’s representation based on surrounding context.

The word “bank” in “the bank approved the loan” and “the river bank was muddy” begins from the same token embedding if tokenized identically but can acquire different contextual representations. Unlike static word embeddings such as Word2Vec or GloVe, Transformer hidden states vary with sentence context.

6. Positional Information

Self-attention does not encode word order by itself. A Transformer therefore injects information about token positions. In the original architecture, sinusoidal positional encodings are added to input embeddings:

PE(pos, 2i) = sin(pos / 10000^(2i / d_model))

PE(pos, 2i + 1) = cos(pos / 10000^(2i / d_model))

Here pos is the token position, i is the dimension index, and d_model is the embedding dimension. Sinusoidal functions use a spectrum of wavelengths to represent position. Other architectures adopt learned absolute position vectors, relative biases, or Rotary Position Embeddings (RoPE), each with different extrapolation and computational properties.

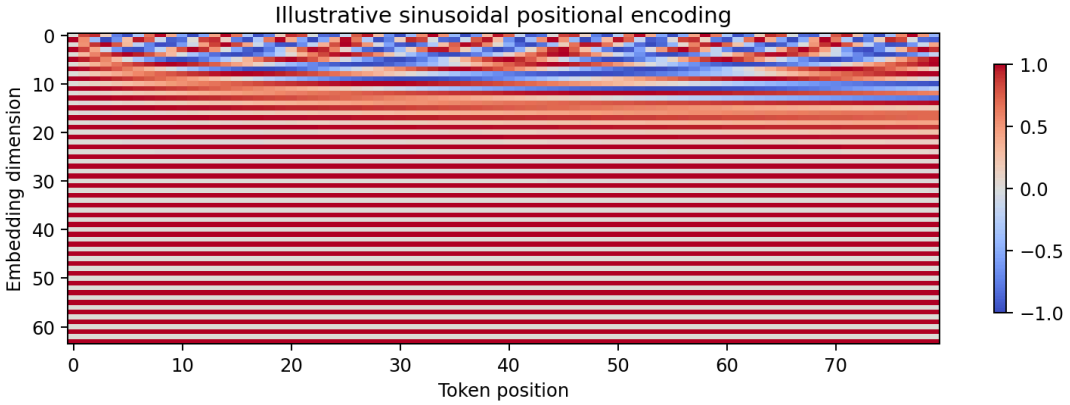


Figure 3. Illustrative sinusoidal positional patterns across positions and dimensions.

7. Self-Attention: Query, Key, and Value

Attention learns how strongly each token should incorporate information from other tokens. From a matrix of token representations X, separate learned projections produce queries Q = XW_Q, keys K = XW_K, and values V = XW_V. Query–key dot products estimate compatibility; softmax converts scaled scores into normalized weights; these weights aggregate the value vectors.

$$\text{Attention}(Q, K, V) = \text{softmax}((QK^T / \sqrt{d_k}) + M)\, V$$

The scaling term $\sqrt{d_k}$ helps avoid overly large logits when the key dimension increases. M represents an optional additive mask whose prohibited locations receive large negative scores. In bidirectional self-attention, positions generally attend in both directions; causal self-attention restricts access to the current and previous positions.

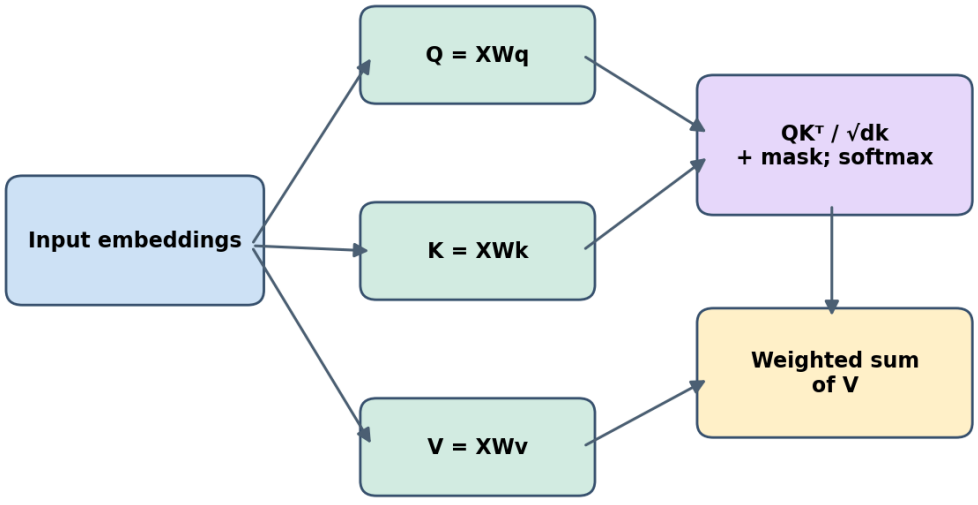


Figure 4. Simplified query–key–value computation for scaled dot-product attention.

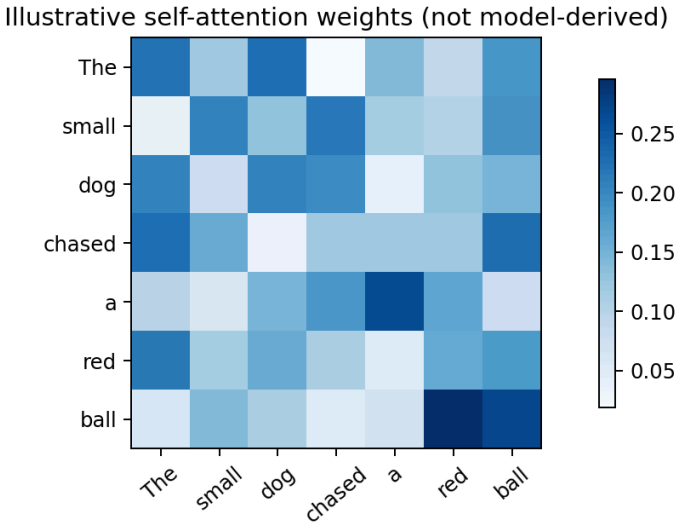


Figure 5. Hypothetical attention distribution; this heatmap is illustrative, not an actual trained model output.

8. Multi-Head Attention

Rather than performing one attention computation, multi-head attention uses several independently learned projections. Each head can focus on different useful relationships, although no fixed linguistic function is guaranteed for a particular head. The head outputs are concatenated and projected back to model dimension:

$$\text{MultiHead}(Q,K,V) = \text{Concat}(\text{head}_1, \ldots, \text{head}\square)\, W_O$$

$$\text{head}_i = \text{Attention}(QW_i^{\vartheta}, KW_i^{K}, VW_i\square)$$

The original base Transformer uses eight attention heads. Recent large language models may use multi-query or grouped-query attention, reducing the number of key/value heads and associated inference memory.

9. Feed-Forward Networks, Residuals, and Normalization

After attention, each token passes through the same position-wise feed-forward network, typically a pair of learned linear transformations with a nonlinear activation. The original formulation was $\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$. Modern variants may use gated activations such as SwiGLU. Residual pathways support information and gradient flow across deep networks, while layer normalization improves optimization stability. The original Transformer applied layer normalization after residual addition; many newer models normalize before each sublayer.

10. Three Main Transformer Families

Architecture	Attention pattern	Typical purposes	Example
Encoder-only	Often bidirectional	Classification, entity extraction, semantic representations	BERT
Decoder-only	Causal self-attention	Autoregressive text and code generation	GPT-family
Encoder–decoder	Encoder self-attention; decoder causal and cross-attention	Translation, summarization, text transformation	T5, BART

Encoder-only models develop rich input representations. Decoder-only models predict successive tokens conditioned on earlier context. Encoder–decoder models transform a source sequence into a target sequence using explicit cross-attention. The choice depends on whether the principal objective is understanding, generation, or input-to-output transformation.

11. Pretraining

Pretraining teaches a model broad patterns using a large and usually diverse corpus before task-specific adaptation. In causal language modeling, the model predicts the next token from prior tokens. In masked language modeling, selected tokens are hidden and reconstructed using allowed context. Denoising objectives, used by models such as T5 and BART, reconstruct deliberately corrupted inputs. These losses shape the general representations stored in the model's learned weights.

Training cycles repeatedly through forward passes, loss computation, backpropagation, and optimizer updates across many batches. Training data quality, filtering, deduplication, model scale, and compute resources all influence outcomes. Pretraining can be expensive but produces reusable weights for downstream tasks.

12. Fine-Tuning and Parameter-Efficient Adaptation

Fine-tuning begins from pretrained weights and updates the model using a specialized task or domain dataset. For example, a pretrained language model might be adapted to classify customer reviews or follow a particular instruction format. Full fine-tuning updates most or all model weights, which offers flexibility but can require significant memory.

Parameter-efficient fine-tuning (PEFT) updates far fewer parameters. Low-Rank Adaptation (LoRA) freezes a base weight matrix W and learns a low-rank adjustment $\Delta W = BA$, giving $W' = W + BA$. This cuts trainable parameter counts and optimizer memory. Instruction fine-tuning uses prompt–response examples to improve instruction-following behavior. Preference optimization can provide a further post-training stage.

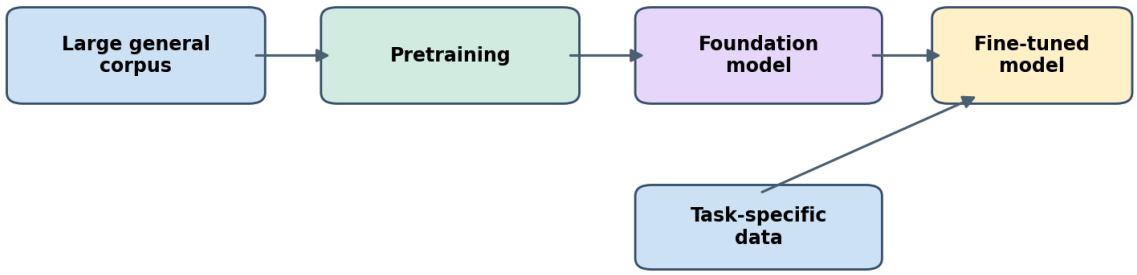


Figure 6. Pretraining creates reusable model weights; fine-tuning adapts them for a specialized task.

13. Pretraining vs. Fine-Tuning

Dimension	Pretraining	Fine-tuning
Goal	Broad transferable capabilities	Specific task or domain adaptation
Starting point	Often random initialization	Pretrained model
Data	Very large, general corpus	Usually smaller, targeted examples

Objective	General prediction or reconstruction	Task labels, instruction responses, preferences
Compute	Typically extremely high	Usually much lower
Result	Foundation model	Adapted model

In a customer-service application, general pretraining creates language capability, while fine-tuning on approved dialogue examples can improve task behavior. Retrieval-Augmented Generation (RAG) complements rather than replaces this approach: it supplies selected external documents at inference time without inherently changing the model's weights.

14. Training vs. Inference

During training, model weights are updated to minimize an objective such as cross-entropy. For a target distribution y and predicted distribution p, $L = -\Sigma_i y_i \log(p_i)$. Optimizers such as AdamW use computed gradients to update weights. During inference, parameters ordinarily remain fixed. Autoregressive models produce one token at a time using decoding methods such as greedy selection, temperature sampling, top-k, top-p, or beam search. Key–value caches avoid repeating some previous attention computations and improve decoding efficiency.

15. Applications

Natural language processing: document classification, information extraction, sentiment analysis, question answering, and summarization. Machine translation: source–target sequence conversion with encoder–decoder architectures. Computer vision: Vision Transformers divide images into patches and process patch embeddings. Speech and audio: recognition, translation, audio understanding, and generation. Software engineering: code completion, program explanation, testing, and assistance with debugging. Multimodal systems combine textual, visual, auditory, or video-derived representations.

16. Advantages and Limitations

Transformers offer flexible contextual modeling, highly parallelizable training across input positions, direct long-distance token interactions, and strong transfer learning. The major challenges are computational requirements, attention memory for long sequences, expensive training and deployment, possible hallucinations and social biases inherited from data, and limited transparency into causal drivers of model decisions. Attention patterns are useful diagnostics but should not be treated as complete explanations of model reasoning.

17. Recent Efficiency Directions

Research areas include efficient implementations such as FlashAttention, sparse or sliding-window attention, key/value cache improvements, quantization, low-rank adaptation, and Mixture-of-Experts systems. Mixture-of-Experts routes tokens to selected feed-forward expert modules, increasing model capacity without activating all parameters for every token. Improvements to long-context handling, multimodal modeling, retrieval, and evaluation remain active areas of development.

18. Conclusion

Transformer architectures combine tokenization, learned embeddings, positional information, attention, feed-forward processing, and normalization to produce powerful representations of sequences. Pretraining provides broad capabilities, and fine-tuning specializes them for downstream tasks. Encoder-only, decoder-only, and encoder–decoder

variants illustrate how the same building blocks support both comprehension and generation. Understanding these mechanisms provides a foundation for studying large language models, generative AI, and multimodal neural systems.

19. References

1. Vaswani, A., et al. (2017). Attention Is All You Need. https://arxiv.org/abs/1706.03762

2. Devlin, J., et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. https://arxiv.org/abs/1810.04805

3. Raffel, C., et al. (2020). Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. https://arxiv.org/abs/1910.10683

4. Hu, E. J., et al. (2022). LoRA: Low-Rank Adaptation of Large Language Models. https://arxiv.org/abs/2106.09685

5. Hugging Face. Transformers documentation: Training and fine-tuning. https://huggingface.co/docs/transformers/training

6. Alammar, J. The Illustrated Transformer. https://jalammar.github.io/illustrated-transformer/